



Universidad
Nacional
de Loja

Universidad Nacional de Loja

FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS, Y LOS RECURSOS NATURALES NO RENOVABLES

Carrera de Ingeniería en Computación

TRABAJO AUTÓNOMO

ASIGNATURA:

Matematicas Discretas

INTEGRANTES:

Yuleisy Cecibel Jaramillo

Josselin Estefania Nero Fernandez

Wilson Joel Guevara Moncada

Sdier Emanuel Roblez.

TÍTULO:

TRABAJO AUTÓNOMO DE TEORÍA DE ÁRBOLES Y RECORRIDOS.

Loja – Ecuador 2026

Introducción: El Árbol como Arquitectura de la Información

En el estudio de las matemáticas discretas, solemos buscar formas de representar las conexiones que rigen nuestro mundo. Dentro de la **teoría de grafos**, surge una estructura cuya elegancia reside en su aparente simplicidad: **el árbol**. Aunque el término evoca imágenes de la naturaleza, en el contexto computacional, un árbol es un **caso especial de grafo** que se define formalmente como una estructura **no dirigida, conexa y que carece de circuitos o ciclos**. Esta ausencia de ciclos no es un detalle menor; es la propiedad fundamental que garantiza que exista un **único paseo o camino entre cualquier par de vértices**, eliminando cualquier ambigüedad en el acceso a la información y permitiendo una navegación jerárquica sumamente precisa.

La **trascendencia de los árboles** en las estructuras de datos modernas es prácticamente incalculable, ya que representan el corazón invisible de nuestra interacción diaria con la tecnología. Por ejemplo, casi todos los **sistemas operativos** actuales dependen de estructuras de árbol para organizar sus sistemas de archivos, permitiéndonos localizar un documento específico entre millones en fracciones de segundo. Más allá del almacenamiento, los árboles son esenciales para la **modelación de procedimientos** que requieren secuencias de decisiones lógicas y para la construcción de **algoritmos de búsqueda** que definen la eficiencia de los motores de datos actuales. Sin esta organización jerárquica, la gestión masiva de datos que define nuestra era sería una tarea caótica e inviable.

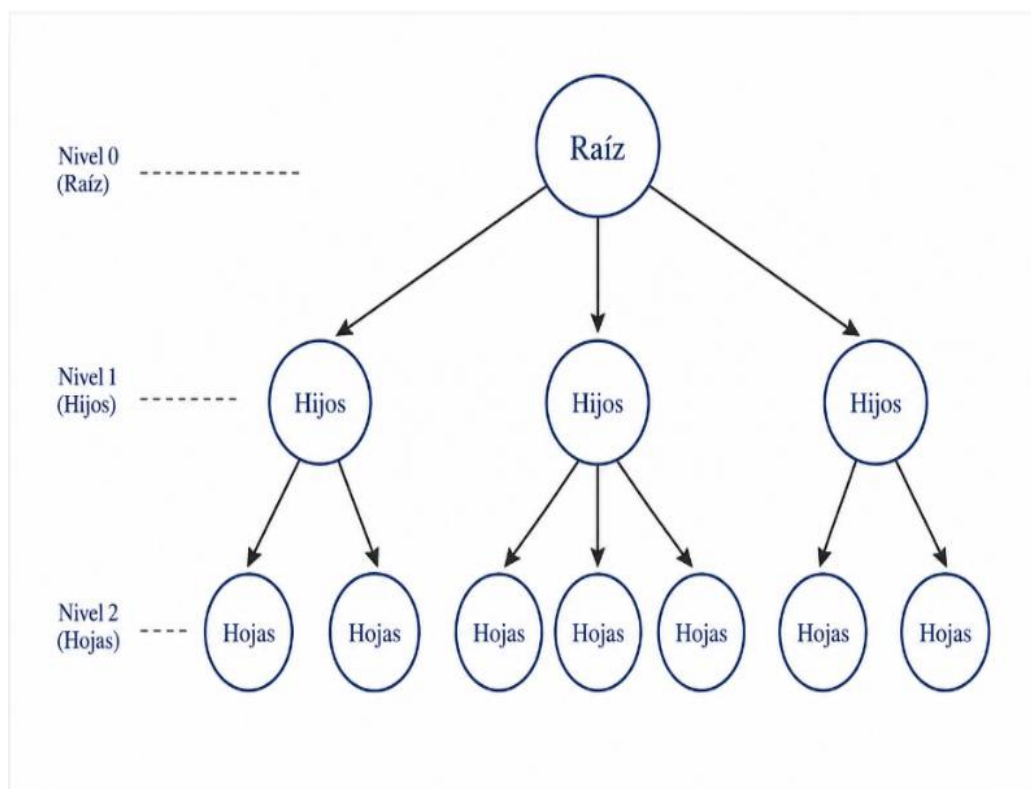
El presente ensayo tiene como propósito desglosar los pilares fundamentales que sostienen esta teoría. En la primera parte (**Parte A**), exploraremos las definiciones y propiedades básicas que caracterizan a los árboles, con un énfasis especial en los **árboles enraizados**, donde un nodo principal actúa como origen y dicta la jerarquía de todos sus descendientes. Posteriormente, en la **Parte B**, analizaremos el concepto de **optimización y búsqueda**, profundizando en cómo la **longitud de paseo** y el diseño de **códigos de prefijo** —como el ingenioso **Código Huffman**— permiten comprimir y transmitir datos de manera eficiente basándose en su frecuencia de aparición. Asimismo, examinaremos la lógica matemática que permite a los **Árboles de Búsqueda Binaria (ABB)** localizar elementos mediante comparaciones rápidas y sistemáticas.

Finalmente, el trabajo abordará la dimensión procedimental en la **Parte C**, mediante el análisis de los **algoritmos de recorrido: preorden, enorden y postorden**. A través de la resolución detallada de ejercicios prácticos, demostraremos que no basta con almacenar datos en una estructura perfecta; es vital comprender la metodología para visitarlos y procesarlos de forma ordenada. Con este recorrido,

se pretende consolidar al árbol no solo como un objeto de estudio matemático, sino como la arquitectura fundamental que dota de inteligencia y velocidad a nuestras máquinas.

Parte A: Fundamentos y Tipos de Árboles

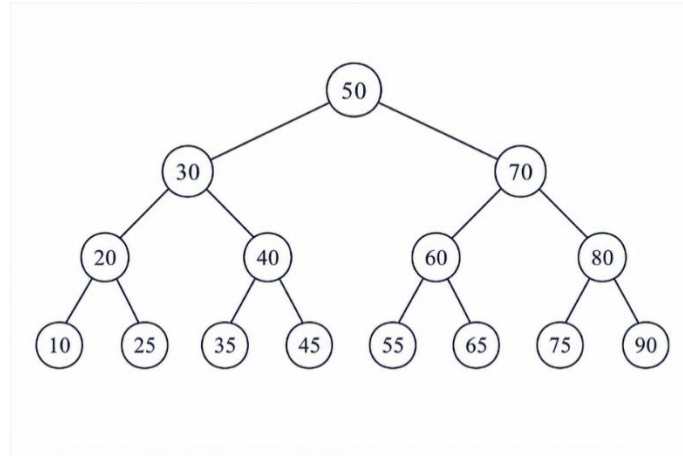
Las matemáticas discretas a veces asustan con sus nombres raros, pero en realidad buscan formas fáciles de resolver problemas del día a día. En el libro de los autores Villalpando y García, el capítulo 7 nos habla de una herramienta genial llamada **árboles** (Villalpando Becerra, 2014, cap. 7). En el mundo de la tecnología, un árbol no es una planta con hojas; es una forma de organizar y conectar información para que las computadoras puedan encontrar cosas en un abrir y cerrar de ojos (Villalpando Becerra, 2014, cap. 7). En este ensayo te explicaré de forma muy clara qué tipos de árboles existen y para qué sirven.



Ejercicio: Un árbol enraizado tiene altura 3 (niveles 0,1,2). La raíz tiene 2 hijos, y cada uno de esos hijos tiene 3 hijos. ¿Cuántos nodos tiene en total y cuál es la complejidad de recorrerlo?

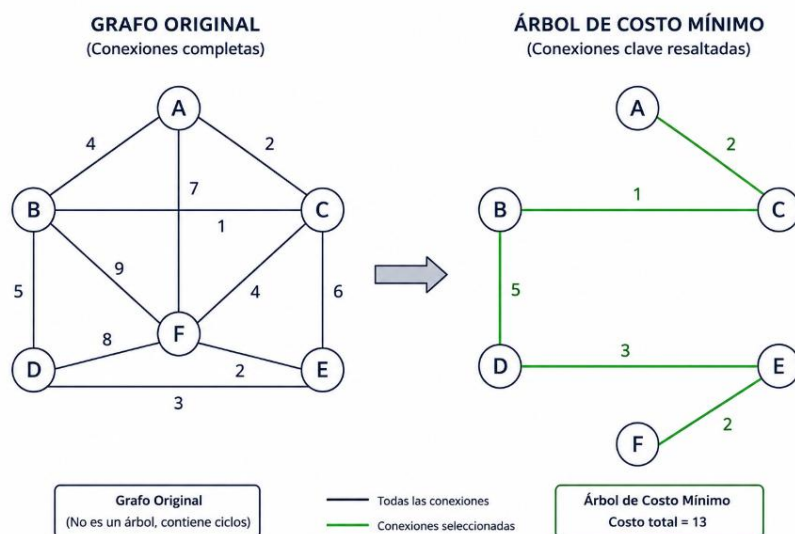
Solución: Nivel 0: 1 nodo (raíz). Nivel 1: 2 nodos. Nivel 2: cada uno de los 2 hijos tiene 3 hijos $\rightarrow 2 \times 3 = 6$ nodos. Total = $1 + 2 + 6 = 9$ nodos. Altura = 3. Recorrer el árbol (por ejemplo, en BFS) toma $O(n)$ tiempo, donde n es el número total de nodos.

- **Árboles de Búsqueda Binaria:** Son perfectos para guardar datos y encontrarlos rápidamente. Su propiedad fundamental consiste en que los valores menores se ubican en el subárbol izquierdo y los mayores en el derecho, lo que permite realizar búsquedas de manera eficiente



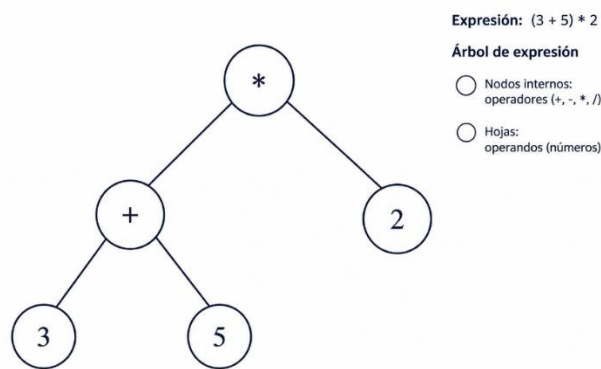
- **Ejercicio:** Construye un árbol ternario ($m=3$) de altura 3 (niveles 1 a 3) completo. ¿Cuántos nodos hoja tiene? Formula general para hojas en árbol m -ario de altura h .
Solución: Un árbol ternario de altura 3 completo tiene niveles 1, 2 y 3. Nivel 1: 1 nodo (raíz); nivel 2: 3 hijos; nivel 3: cada uno de esos 3 tiene 3 hijos = 9 nodos. Así, hojas = 9. En general, un árbol m -ario completo de altura h (niveles 1 a h) tiene m^{h-1} hojas. Recorrer el árbol completo lleva $O(n)$ tiempo.

- **Árboles Generadores Mínimos:** Permiten conectar todos los vértices de un grafo utilizando el menor costo posible y evitando ciclos, siendo ampliamente utilizados en problemas de diseño de redes y optimización (Villalpando Becerra, 2014, cap. 7).



- **Ejercicio:** Inserta en un BST vacío la secuencia de claves [8, 3, 10, 1, 6, 14, 4, 7] y muestra el recorrido in-orden resultante. ¿Cuál es la complejidad de búsqueda en este árbol?
- Solución:** El árbol resultante (dibujado) tendrá 8 como raíz, 3 a la izquierda, 10 derecha, etc. El recorrido in-orden devuelve las claves ordenadas: 1, 3, 4, 6, 7, 8, 10, 14. En un BST balanceado, buscar una clave toma $O(\log n)$ tiempo. En el peor caso (árbol degenerado), sería $O(n)$.

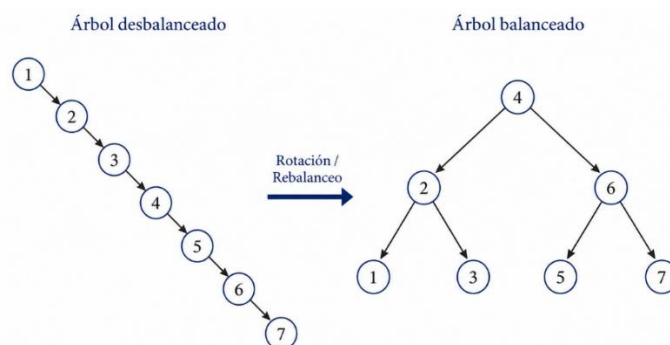
Árboles de Expresión: Representan expresiones algebraicas mediante una estructura jerárquica donde los operadores corresponden a nodos internos y los operandos a hojas, facilitando la evaluación correcta de las operaciones según su precedencia (Villalpando Becerra, 2014, cap. 7).



Ejercicio: Construya el árbol de expresión para la fórmula $(3 + 5) * 2$ y evalúelo.

Solución: El árbol tiene $*$ como raíz, subárbol izquierdo con $+$ y hojas 3,5; subárbol derecho es la hoja 2. La evaluación (post-orden) calcula $3+5=8$, luego $8*2=16$. Los árboles de expresión permiten definir jerarquía de operaciones en compiladores.

- **Árboles Balanceados (AVL):** A veces, al meter muchos datos, el árbol se empieza a inclinar mucho hacia un solo lado y se vuelve lento. Los árboles AVL tienen un truco: se "equilibran" solos de forma automática para que todos sus lados midan lo mismo y sigan funcionando al máximo de su velocidad.



- **Ejercicio:** Inserte en un árbol AVL vacío los valores [30, 20, 40, 10, 25, 50, 22] y muestre las rotaciones realizadas. ¿Cuál es la altura final?

Solución: Tras insertar 30,20,40, el árbol está balanceado. Al insertar 10, se desequilibra un poco (factor -2 en 30) y se realiza rotación simple derecha; luego insertar 25,50 no produce desbalances fuertes. Finalmente, insertar 22 genera un caso Doble (rotación izquierda-derecha). El árbol resultante tiene altura $O(\log n)$ (aquí 3 niveles). Las operaciones de inserción y balanceo en AVL toman $O(\log n)$ tiempo.

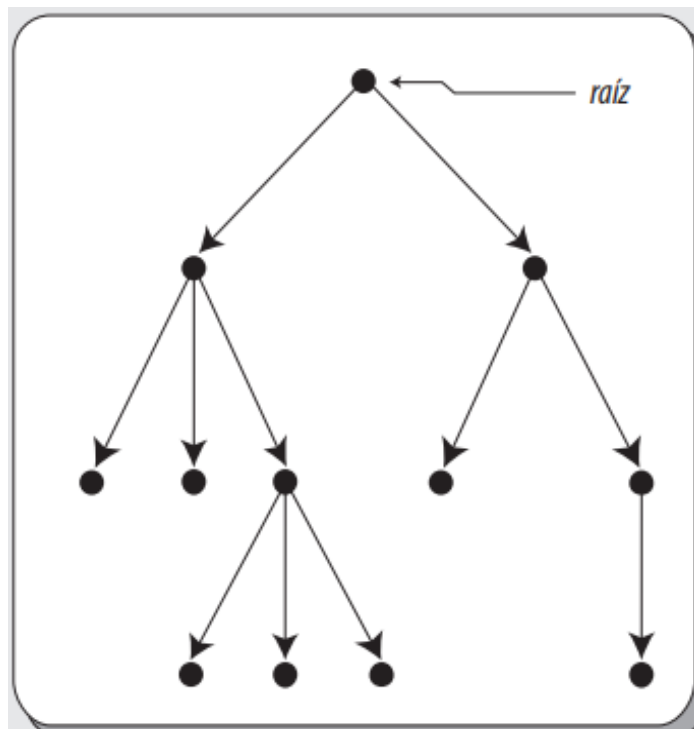
+

ÁRBOLES ENRAIZADOS

Definición de árboles enraizados y condiciones.

A diferencia de un árbol común donde ninguno de sus nodos es más importante, en un árbol enraizado existe lo que es la raíz que es un nodo con la característica de que “la valencia de entrada tiene que ser 0 y las valencias de entrada de los otros nodos tienen que ser 1” (Villalpando Becerra, 2014).

Esto quiere decir que al nodo raíz (valencia 0) es el único que no le llega ninguna flecha o conexión de entrada, pero a los demás si tiene como entrada una valencia de 1 lo que garantiza que no haya caminos cerrados o circuitos, al tener un único padre, pero ese no siempre es la raíz, formando así una jerarquía.



Para poder entender mejor un árbol enraizado y la jerarquía de los nodos se los dibuja como están en esta imagen, es decir, la raíz en la parte superior.

Clasificación de vértices:

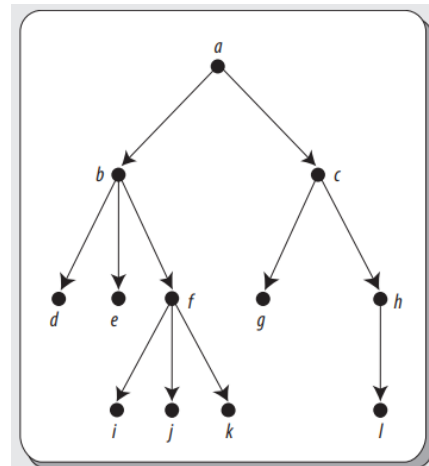
Otro punto a señalar es cuando los nodos tienen una valencia específica:

- **Hoja o nodo terminal:** Un nodo se lo considera una hoja cuando “la valencia de salida es cero” (Villalpando Becerra, 2014).

POR EJEMPLO: En la imagen las hojas serían d, e, g, i, j, k, l.

- **Rama, nodo rama o nodo interno:** En esta parte un nodo se lo considera rama cuando su “valencia de salida es diferente a cero” (Villalpando Becerra, 2014).

POR EJEMPLO: En la imagen las ramas serían los vértices a, b, c, f, h.



Relaciones Jerárquicas

Existen nombres específicos para determinar las relaciones entre los vértices de un árbol enraizado entre ellas son:

- **Padre e Hijo:** “Se dice que un vértice b es un hijo de a si existe un lado dirigido del vértice a al vértice b” (Villalpando Becerra, 2014). Esto quiere decir que el padre de b es a.
- **Hermanos:** Por otro lado, dos vértices o más hijos del mismo padre se lo consideran hermanos
- **Descendientes:** Se dice que un vértice c es un descendiente del vértice a si existe un paseo dirigido del vértice a al vértice c. Por ejemplo, a es padre de b y b es padre de c, por lo tanto, c es descendiente de a.
- **Ancestro:** Si c es descendiente de a, entonces a es ancestro de c.

POR EJEMPLO:

La Raíz es a.

HIJOS y PADRES:

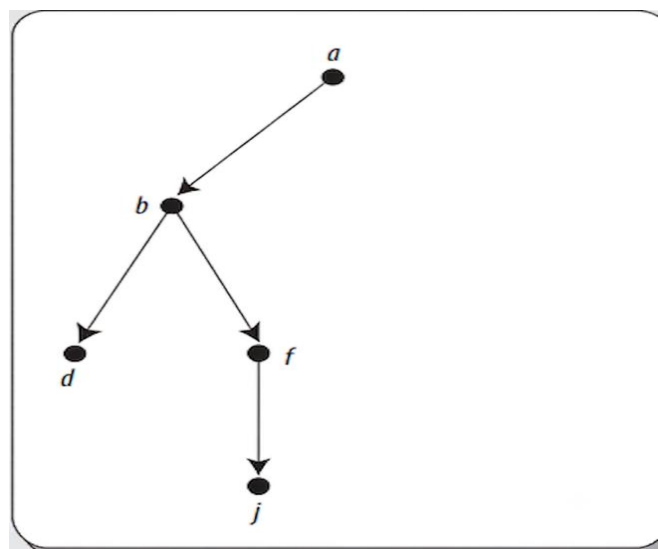
- b es hijo de a. (a es padre de b).
- d y f son hijos de b (b es padre de d y f).
- j es hijo de f (f es padre de j).

HERMANOS:

- d y f son hermanos

DESCENDIENTES:

- a: b, d, f y j.



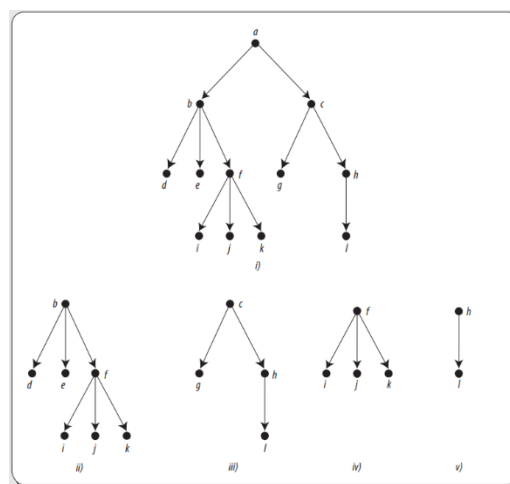
- b: d, f y j.
- f: j.

ANTECEDENTES:

- j: f, b y a.
- f y d: b y a.
- b: a.

Subárbol:

“Si tenemos un árbol enraizado y tomamos un nodo rama, el subárbol con raíz es un **subgrafo** donde el conjunto de vértices incluye al nodo y a todos sus descendientes” (Villalpando Becerra, 2014).



Es importante aclarar que para un árbol dado existen tantos subárboles como nodos rama tenga el árbol.

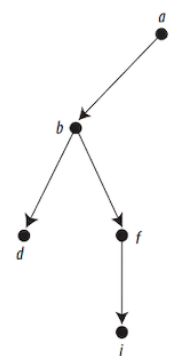
Parte B (Optimización y Búsqueda): LONGITUD DE PASEO:

Es necesario determinar la cantidad de lados que existen desde la raíz del árbol, esto se lo denomina longitud.

La longitud de un paseo para un vértice en un árbol enraizado es el número de lados en el paseo desde la raíz hasta el vértice.

Algo importante es que la longitud de la raíz es 0.

Por ejemplo: La longitud del vértice j en relación a la raíz es de 3.



Altura del Árbol

“La altura de un árbol enraizado (h) es la máxima de las longitudes de los paseos de un árbol y se puede denotar como $h(T)$ ” (Villalpando Becerra, 2014).

Tomando el ejemplo anterior donde todas las longitudes son:

Vértice	Longitud
a	0
b	1
d	2
f	2
j	3

Como podemos ver en la tabla la longitud del paseo más grande es de j por lo tanto la altura del árbol es 3.

CÓDIGO DE PREFIJOS

Definición de Prefijos

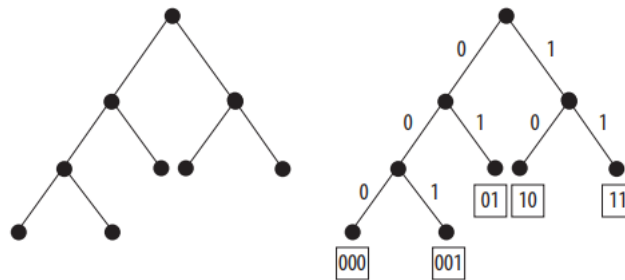
Un código de prefijos no es más que un conjunto de sucesiones donde se tiene que cumplir una regla fundamental: ninguna sucesión del conjunto puede ser el prefijo de otra. Esto para que no haya ambigüedad.

Uso de Árboles Binarios

Existe una correspondencia biunívoca (cada código de prefijo le pertenece a un paseo donde termina en una sola hoja del árbol).

Para crear el código se etiqueta cada rama: 0 para el camino a la izquierda y 1 para el camino de la derecha. De esta manera tenemos el código que se formara al final de la hoja del árbol y que representa el paseo desde la raíz hasta la hoja.

EJEMPLO:



Teniendo el árbol binario se puede construir el código, pero teniendo el código también se puede construir el árbol.

Código de Huffman

Es un esquema de codificación llamado código dependiente de frecuencia. Su objetivo principal es comprimir datos, para reducir el número de bits necesarios para almacenar o transmitir información. Se basa en eliminar la restricción de todos los códigos que tengan la misma longitud; los caracteres más comunes tienen códigos más cortos que los menos comunes.

PASOS:

Paso 1: Para conseguir el código Huffman es necesario sacar frecuencias de aparición (probabilidades) de cada carácter o símbolo que se va a codificar. Y ordenar los valores de menor a mayor.

Paso 2: Agrupar y sumar las dos probabilidades más pequeñas.

Paso 3: Dibujar dos lados (aristas) que unan el nuevo valor con los dos originales que lo generaron.

Paso 4: Marcar las probabilidades ya usadas para no volver a tomarlas y repetir el proceso hasta que solo quede una probabilidad sin marcar, que siempre será igual a 1.00 (esta se convierte en la raíz del árbol).

Paso 5: Una vez construido el árbol binario, se etiquetan los lados: los que van al subárbol izquierdo con 0 y los que van al derecho con 1.

Paso 6: La secuencia de ceros y unos desde la raíz hasta la hoja de cada carácter constituye su código binario único.

EJEMPLO:

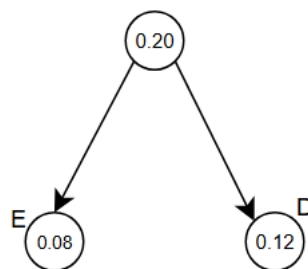
Tabla de probabilidades de aparición

Caracteres	A	B	C	D	E
Probabilidad	0.40	0.25	0.15	0.12	0.08

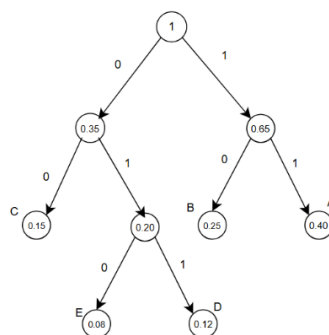
Como podemos observar en la tabla los valores no están ordenados es por esta razón que es necesario ordenar de menor a mayor:

Caracteres	E	D	C	B	A
Probabilidad	0.08	0.12	0.15	0.25	0.40

Ahora podemos identificar de una manera más fácil lo que es las dos probabilidades más bajas por lo que las sumamos lo cual nos va a dar una rama que va a tener este valor obtenido. Como se puede observar en el subgrafo.

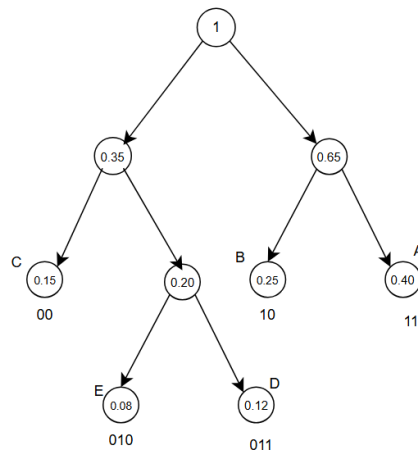


Repetimos este mismo proceso hasta que quede lo que es la raíz con un valor de 1 (la suma de final de todo).



Luego de tener el árbol colocamos 0 en los lados izquierdos y 1 en los lados derechos como podemos ver en la imagen anterior.

Y por último para obtener el código de Huffman ponemos el código binario en cada hoja que representa el paseo de la raíz hasta esa hoja.



De esta manera tenemos los resultados:

Código de Huffman Resultante					
Dígito	A	B	C	D	E
Código	11	10	00	011	010

Árboles de Búsqueda Binaria (ABB)

La lógica matemática de los **Árboles de Búsqueda Binaria (ABB)** es lo que permite que una computadora encuentre un dato entre millones en una fracción de segundo. Esta eficiencia no es casualidad, sino que se basa en un **ordenamiento lineal** estricto y en el concepto de **decisión binaria**.

Aquí te detallo los fundamentos matemáticos que los rigen:

- **La Regla Estructural de Orden:** Para que un árbol binario sea "de búsqueda", debe cumplir una condición matemática en cada uno de sus nodos: si tomamos un nodo con valor X, todos los elementos en su **subárbol izquierdo deben ser menores** que X, y todos los de su **subárbol derecho deben ser mayores**. Esta regla se aplica de forma recursiva, lo que significa que cada nodo rama actúa como un filtro que divide los datos.
- **Procedimiento de Búsqueda por Comparación:** Matemáticamente, la búsqueda en un ABB no es lineal (no revisa uno por uno), sino que utiliza **comparaciones sucesivas**. Al buscar un objeto X, se compara con la raíz:
 1. Si es igual, la búsqueda termina.
 2. Si es menor, la lógica matemática nos obliga a ignorar todo el lado derecho y movernos al hijo izquierdo.

3. Si es mayor, descartamos el lado izquierdo y vamos al derecho. Este proceso continúa hasta hallar la clave o alcanzar una **hoja**, lo que indicaría que el dato no existe.
- **Relación con el Recorrido "Inorden":** Existe una propiedad matemática muy interesante: si realizas un recorrido de tipo **Inorden** (Izquierdo-Nodo-Derecho) en un ABB, los elementos se obtienen siempre en un **orden lineal lógico** (de menor a mayor). Esto demuestra que la estructura jerárquica del árbol preserva perfectamente la relación de orden de los datos originales.
 - **Eficiencia y Complejidad Logarítmica:** La gran ventaja matemática es su velocidad. En un árbol que está bien equilibrado (como los árboles **AVL**), la complejidad de la búsqueda es **logarítmica $O(\log n)$** . Esto significa que, en lugar de dar n pasos (como en una lista común), solo damos unos pocos pasos proporcionales a la altura del árbol, descartando aproximadamente la mitad de los datos en cada movimiento.

Sean las claves {6, 8, 10, 12, 14, 15, 18} y sean los árboles T_1 y T_2 de la figura 7.17. En este caso, el árbol T_1 es un árbol de búsqueda binaria para dichas claves, mientras que el árbol binario T_2 no es de búsqueda binaria, ya que si se considera el elemento 10, todos los elementos del subárbol izquierdo son menores; sin embargo, no todos los elementos del subárbol derecho son mayores, ya que en este, el elemento 6 es menor que 10 y debería ir en el subárbol izquierdo.

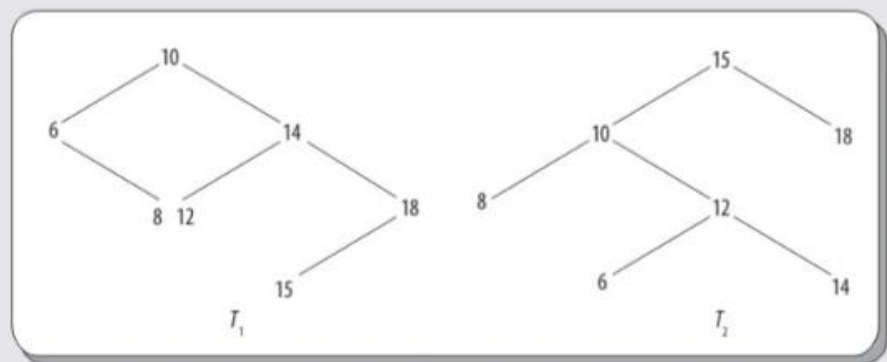


Figura 7.17 El árbol T_1 es un árbol de búsqueda binaria y T_2 es solo un árbol binario.

Parte C (Recorridos - Ejercicios Prácticos):

Los recorridos en un árbol

Los recorridos en un árbol binario son métodos sistemáticos para visitar cada nodo exactamente una vez, lo cual es fundamental para acceder y procesar los datos almacenados en su estructura. De acuerdo con las fuentes, estos recorridos se clasifican principalmente como **recorridos en profundidad**, los cuales se dividen en tres tipos esenciales: **Preorden**, **Inorden** (referido como *Enorden* en el texto) y **Postorden**.

A continuación, se presenta la explicación analítica y la resolución paso a paso de cada uno:

1. Recorrido en Preorden (NID)

Este recorrido sigue la secuencia **Nodo-Izquierdo-Derecho**. En este método, se procesa la raíz antes que sus subárboles.

Pasos fundamentales:

1. **Visitar el nodo raíz (N).**
2. **Recorrer el subárbol izquierdo (I)** en preorden.
3. **Recorrer el subárbol derecho (D)** en preorden.

Resolución paso a paso (Ejemplo Figura 7.30):

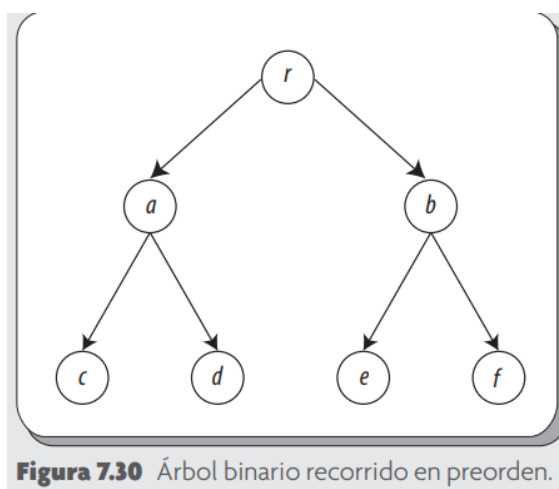


Figura 7.30 Árbol binario recorrido en preorden.

- **Inicio:** Se visita la raíz del árbol, que es el nodo **r**.
- **Subárbol Izquierdo:** Se aplica el orden NID a los descendientes de la izquierda (**{a, c, d}**). Primero se visita la raíz del subárbol (**a**), luego su hijo izquierdo (**c**) y finalmente su hijo derecho (**d**).
- **Subárbol Derecho:** Se aplica el orden NID a los descendientes de la derecha (**{b, e, f}**). Se visita primero la raíz del subárbol (**b**), seguida del nodo izquierdo (**e**) y el derecho (**f**).
- **Resultado final:** **r, a, c, d, b, e, f**.

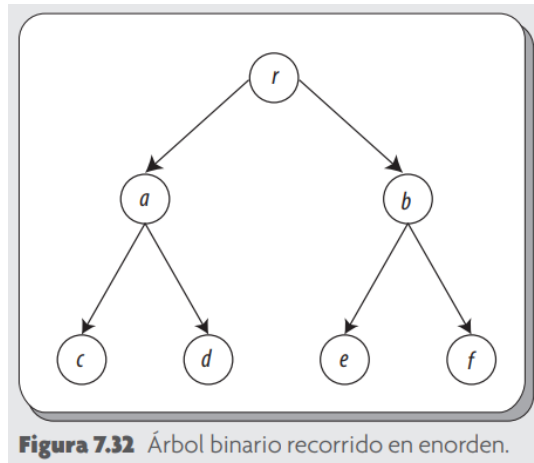
2. Recorrido en Inorden / Enorden (IND)

Este recorrido sigue la secuencia **Izquierdo-Nodo-Derecho**. Es especialmente útil en árboles de búsqueda binaria, ya que permite obtener los elementos en un orden lineal lógico.

Pasos fundamentales:

1. **Recorrer el subárbol izquierdo (I)** en inorden.
2. **Visitar el nodo raíz (N).**
3. **Recorrer el subárbol derecho (D)** en inorden.

Resolución paso a paso (Ejemplo Figura 7.32):



- **Subárbol Izquierdo:** Se procesa primero el lado izquierdo de la raíz. Siguiendo el orden IND para los nodos {a, c, d}, se visita el nodo izquierdo (**c**), luego su raíz (**a**) y después el nodo derecho (**d**).
- **Raíz:** Tras completar el lado izquierdo, se visita la raíz principal del árbol, el nodo **r**.
- **Subárbol Derecho:** Se procesa el lado derecho ({b, e, f}) siguiendo IND. Se visita el nodo izquierdo (**e**), luego su raíz (**b**) y finalmente el nodo derecho (**f**).
- **Resultado final:** c, a, d, r, e, b, f.

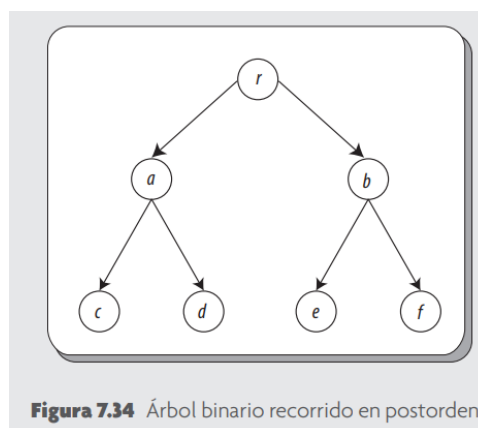
3. Recorrido en Postorden (IDN)

Este recorrido sigue la secuencia **Izquierdo-Derecho-Nodo**. En este método, la raíz es el último elemento en ser visitado.

Pasos fundamentales:

1. **Recorrer el subárbol izquierdo (I)** en postorden.
2. **Recorrer el subárbol derecho (D)** en postorden.
3. **Visitar el nodo raíz (N).**

Resolución paso a paso (Ejemplo Figura 7.34):



- **Subárbol Izquierdo:** Se aplica IDN al conjunto {a, c, d}. Se visita primero el hijo izquierdo (c), luego el hijo derecho (d) y finalmente su raíz (a).
- **Subárbol Derecho:** Se aplica IDN al conjunto {b, e, f}. Se visita el hijo izquierdo (e), seguido del hijo derecho (f) y por último su raíz (b).
- **Raíz:** Una vez recorridos ambos subárboles por completo, se visita la raíz principal del árbol, el nodo r.
- **Resultado final:** c, d, a, e, f, b, r

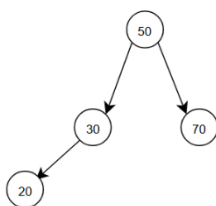
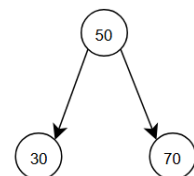
Ejercicio 1:

- **TEMA:** El colapso del sistema de inicio de sesión.
- **CONTEXTO:** Un sistema de base de datos para una plataforma que tiene millones de usuarios, los cuales quieren iniciar sesión, en este proceso el sistema debe buscar el correo electrónico en la base de datos para verificar la contraseña. Para evitar colapsos en la plataforma se organiza la base de datos en un Árbol Binario de Búsqueda (BST). Para lograr estructurar los correos para cada uno se estructura ID numérico único según el orden de creación.
- De esta manera tenemos los siguientes datos:
 - [50, 30, 70, 20, 40, 80, 35, 45, 85, 42]

Estructuración de árbol:

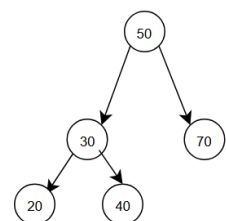
Paso 1: El primer número es 50 (se convierte en raíz).

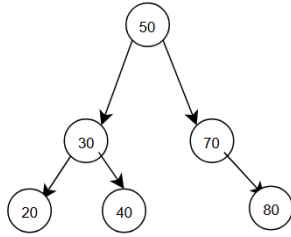
Paso 2: Los dos siguientes valores son 30 y 70, $30 < 70$, por lo tanto 30 va en la izquierda y 70 a la derecha.



Paso 3: Luego tenemos el 20 que analizando tenemos $20 < 50$ y $20 < 30$ por lo tanto queda de la siguiente manera.

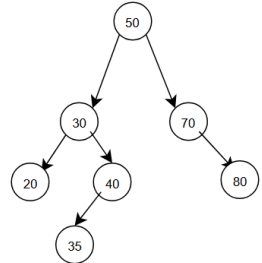
Paso: El siguiente número es 40 analizando tenemos que $40 < 50$ por lo tanto va en el lado izquierdo comparamos con 30 siendo $40 > 30$ por lo tanto va en el lado derecho.



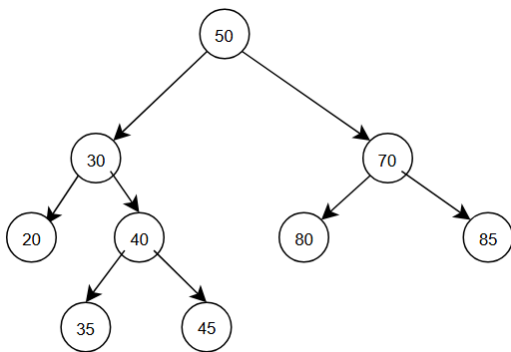
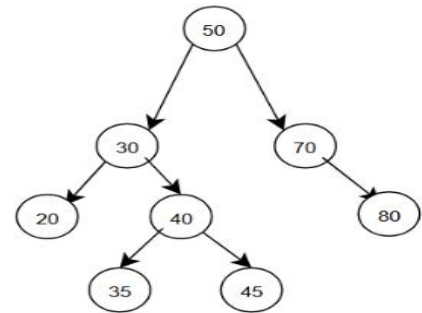


- Paso 5: El siguiente número es 80 como $80 > 50$ va en el lado derecho, y luego $80 > 70$ por lo que también va de lado derecho.

Paso 6: $35 < 50$ (va de lado izquierdo), $35 > 30$ (va de lado derecho) y $35 < 40$ (va de lado izquierdo).

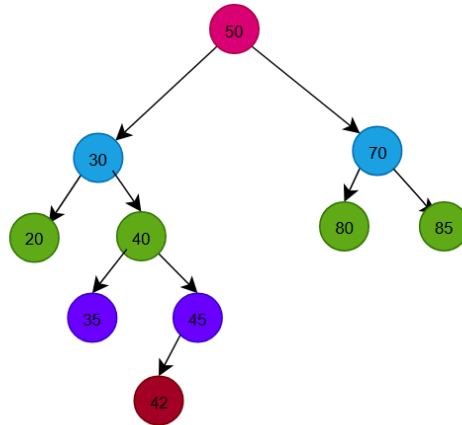


Paso 7: $45 < 50$ (izquierdo), $45 > 30$ (derecho) y $45 > 40$ (derecho).



Paso 8: $85 > 50$ (derecho), $85 > 70$ (derecho). Pero aquí sucede algo el 80 ya está en lado derecho es por esa razón que de los dos tenemos que definir de 80 y 85 por eso como $80 < 85$ pasa a la izquierda y 85 se mantiene en la derecha.

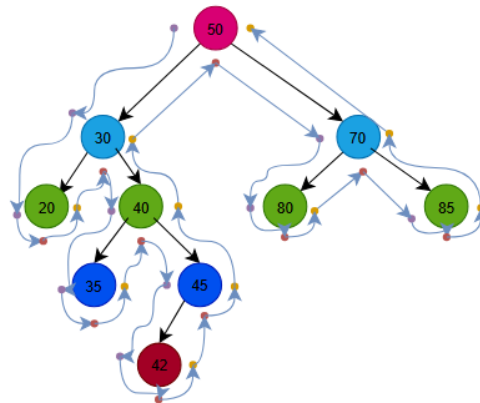
Paso 9: $42 < 50$ (izquierdo), $42 > 30$ (derecho), $42 > 40$ (derecho), $42 < 45$ (izquierdo). Quedando así el árbol.



Recorridos:

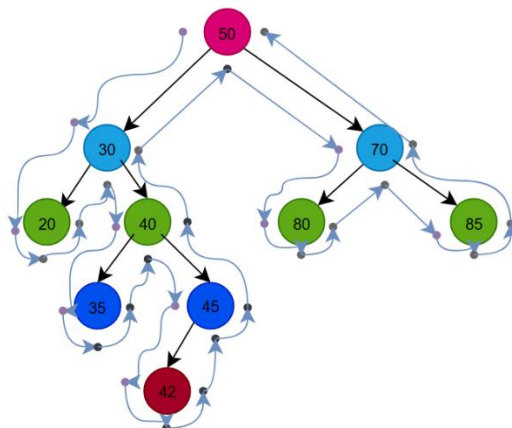
Paso 1: Para definir los recorridos es necesario poner puntos de guía por cada uno de los vértices o nodos vamos a poner tres puntos uno a la izquierda (preorden), otro a la derecha (postorden) y uno en la mitad (inorden).

Paso 2: Empezamos por la raíz en el punto de preorden (izquierda) y seguimos uniendo los puntos a medida que se avanza a modo ir recorriendo todos los puntos.



PREORDEN:

Para definir la secuencia de preorden se deja los puntos de la izquierda y se realiza se sigue las flechas:

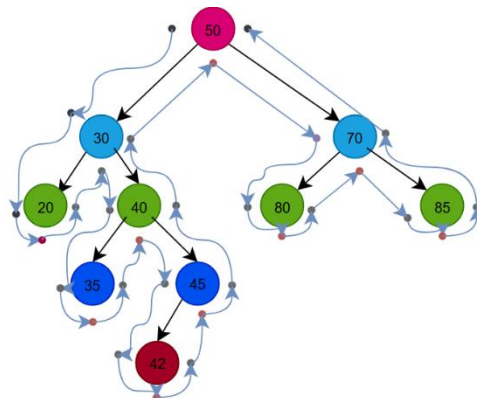


PRE= {50, 30, 20, 40, 35, 45, 42, 70, 80, 85}

El preorden sirve para guardar y compartir el grafo, cuando guardamos a la información de un grafo se convierte en un archivo plano, es por eso que este orden no facilita su reconstrucción y evita la pérdida de información.

Inorden

Para el Inorden se deja solo los puntos del medio se sigue el recorrido siguiendo las flechas.

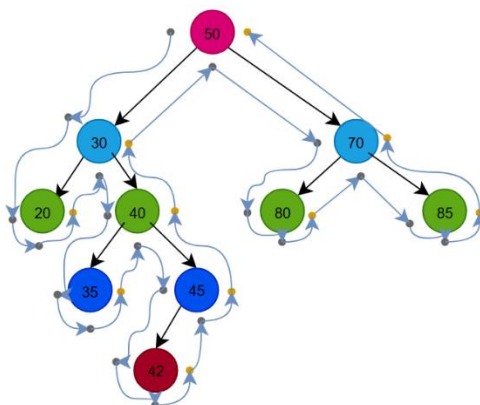


In = [20,30,35,40,42,45,50,70,80,85]

El inorden sirve para recuperar y exportar la información del árbol de manera perfectamente organizada de menor a mayor, lo que permite generar reportes, listados y consultas clasificadas directamente desde el grafo sin tener que gastar memoria ni tiempo de procesamiento del procesador en algoritmos de ordenación externos adicionales. Cuando una base de datos utiliza un BST, este recorrido facilita la extracción de datos de forma secuencial y limpia, sirviendo además como el método de validación por excelencia para comprobar que el árbol mantiene su estructura lógica y sus propiedades de búsqueda intactas.

Postorden

Para el Postorden se deja solo los puntos de la derecha se sigue el recorrido siguiendo las flechas.



POST = [20,35,42,45,40,30,85,80,70,50]

El postorden sirve para liberar la memoria física y eliminar de manera segura la estructura del grafo de abajo

hacia arriba, ya que al procesar siempre los nodos hijos antes que, a los nodos padres, permite que los sistemas operativos borren los datos de las hojas sin dejar referencias rotas o punteros "huérfanos" en el sistema. En la práctica de la computación, este recorrido es el estándar por excelencia para los algoritmos destructores de árboles y para la traducción de operaciones matemáticas dentro de compiladores (notación polaca inversa), asegurando que las dependencias inferiores se resuelvan por completo antes de ejecutar o eliminar la raíz principal.

EJERCICIO 2

Recorrido en Preorden En este recorrido, el enfoque principal es visitar el nodo padre antes de procesar sus ramas. Es sumamente útil para clonar o copiar un árbol estructuralmente.

Paso 1: Visitar el nodo **Raíz**.

Paso 2: Recorrer el subárbol **Izquierdo** en preorden.

Paso 3: Recorrer el subárbol **Derecho** en preorden.

2. Recorrido en Inorden (o Enorden) Como su nombre lo indica, la raíz se visita entre los subárboles. En un árbol binario de búsqueda, este recorrido es fundamental porque devuelve los valores ordenados de forma ascendente.

El libro de Villalpando Becerra se refiere a este método como "Recorrido en enorden".

Paso 1: Recorrer el subárbol **Izquierdo** en inorden.

Paso 2: Visitar el nodo **Raíz**.

Paso 3: Recorrer el subárbol **Derecho** en inorden.

3. Recorrido en Postorden Aquí, la raíz es lo último que se evalúa.

Se utiliza frecuentemente para eliminar árboles (ya que debes borrar los hijos antes que el padre) y para evaluar expresiones matemáticas (como se menciona en el tema "Árboles de expresión" del libro).

Paso 1: Recorrer el subárbol **Izquierdo** en postorden.

Paso 2: Recorrer el subárbol **Derecho** en postorden.

Paso 3: Visitar el nodo **Raíz**.

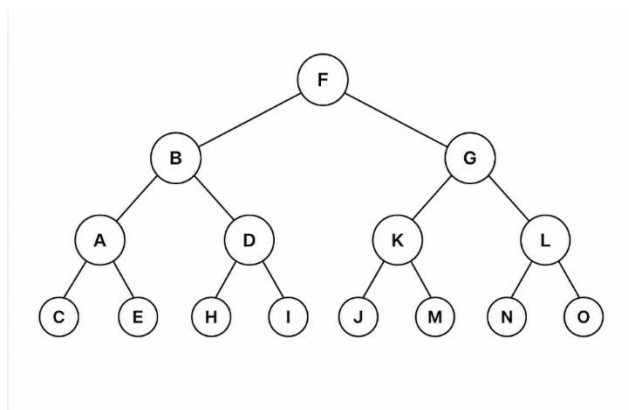
Análisis Teórico de los Recorridos Fundamentales

Un árbol binario es una estructura de datos no lineal donde cada nodo tiene, como máximo, dos hijos (izquierdo y derecho). Recorrer un árbol significa visitar cada uno de sus nodos exactamente una vez. La diferencia entre los métodos radica en el orden lógico en que se visita la "raíz" en relación con sus subárboles.

Ejercicio 1: Árbol Alfabético (Estructura de Búsqueda)

propone la construcción de un árbol binario de cuatro niveles de altura estructurado bajo un criterio **lexicográfico** (alfabético). Este modelo no es arbitrario; se rige por la propiedad fundamental de los árboles binarios de búsqueda: para cualquier nodo dado V, todos los elementos de su subárbol izquierdo son menores que V (antepuestos en el abecedario), y todos los elementos de su subárbol derecho son mayores que V (pospuestos en el abecedario).

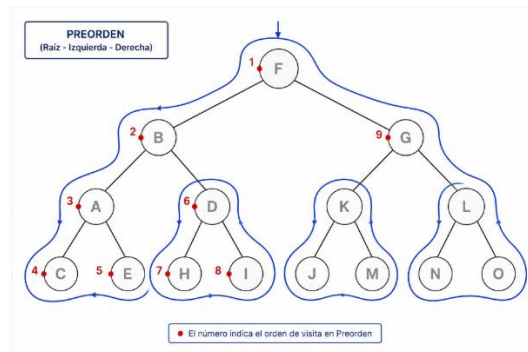
Estructura del Árbol:



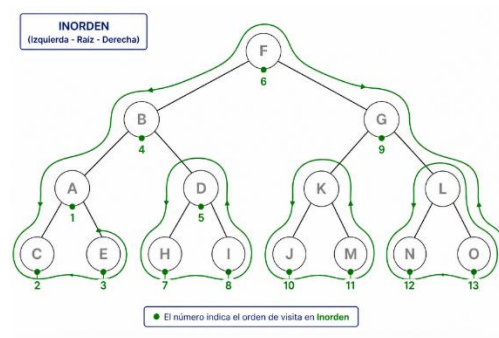
- Nivel 1: F (Raíz principal)
- Nivel 2: B (Hijo izquierdo de F), G (Hijo derecho de F)
- Nivel 3: A (Hijo izquierdo de B), D (Hijo derecho de B), K (Hijo izquierdo de G), L (Hijo derecho de G)
- Nivel 4: C (Hijo izquierdo de A), E (Hijo derecho de A), H (Hijo izquierdo de D), I (Hijo derecho de D), J (Hijo izquierdo de K), M (Hijo derecho de K), N (Hijo izquierdo de L), O (Hijo derecho de L)

Resolución Analítica:

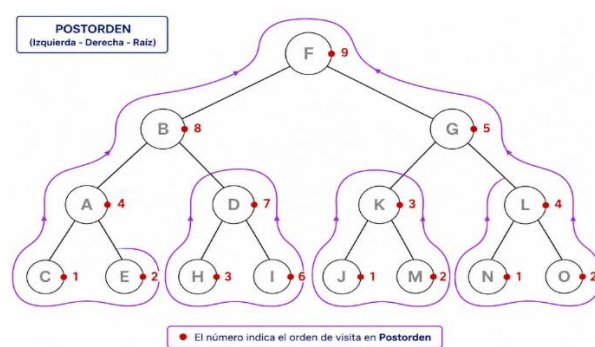
- **Preorden (Raíz, Izq., Der):** F, B, A, D, C, E, G, I, H



- **Inorden (Izq., Raíz, Der):** A, B, C, D, E, F, G, H, I (Nota que quedan en orden alfabético).



- **Postorden (Izq., Der, Raíz):** A, C, E, D, B, H, I, G, F

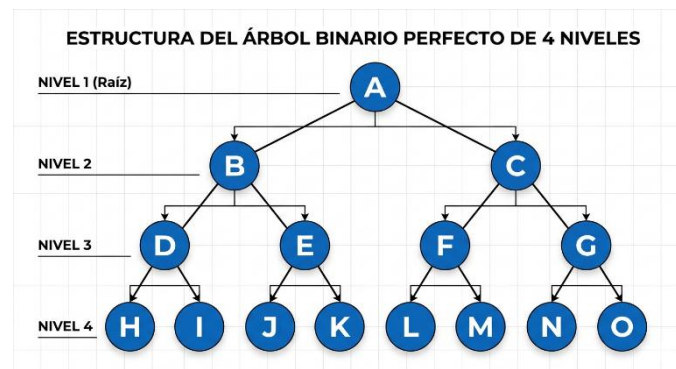


EJERCICIO 3

El ejercicio utiliza un árbol binario balanceado (perfecto) de 4 niveles con un total de 15 nodos.

- **Nivel 1 (Raíz):** A
- **Nivel 2:** B (hijo izquierdo), C (hijo derecho)

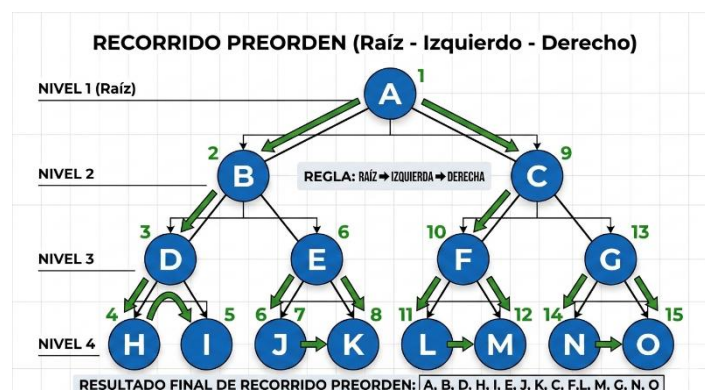
- **Nivel 3:** D, E (hijos de B); F, G (hijos de C)
- **Nivel 4:** H, I (hijos de D); J, K (hijos de E); L, M (hijos de F); N, O (hijos de G)



1. Recorrido Preorden (Raíz - Izquierdo - Derecho)

En este recorrido se procesa primero la raíz, luego se visita por completo el subárbol izquierdo y al final el subárbol derecho.

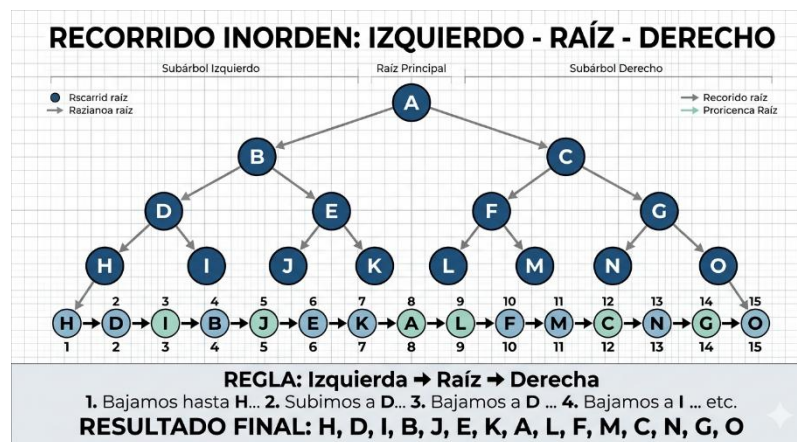
- **Regla:** Raíz → Izquierda → Derecha
- **Explicación del desarrollo:**
 1. Comenzamos en la raíz principal (**A**).
 2. Bajamos por la izquierda al nodo (**B**).
 3. Continuamos a la izquierda al nodo (**D**).
 4. Llegamos al nodo más izquierdo del último nivel (**H**).
 5. Regresamos un paso para visitar el hermano derecho (**I**).
 6. Subimos y nos movemos al subárbol derecho de B, procesando (**E**), luego su izquierdo (**J**) y su derecho (**K**).
 7. Repetimos el mismo orden lógico para todo el lado derecho del árbol comenzando en (**C**).
- **Resultado final:** A, B, D, H, I, E, J, K, C, F, L, M, G, N, O



2. Recorrido Inorden (Izquierdo - Raíz - Derecho)

En este recorrido se visita primero el subárbol izquierdo, luego se procesa la raíz y al final el subárbol derecho. En un árbol binario perfecto, este recorrido tiene la particularidad de leer todos los nodos visualmente de izquierda a derecha.

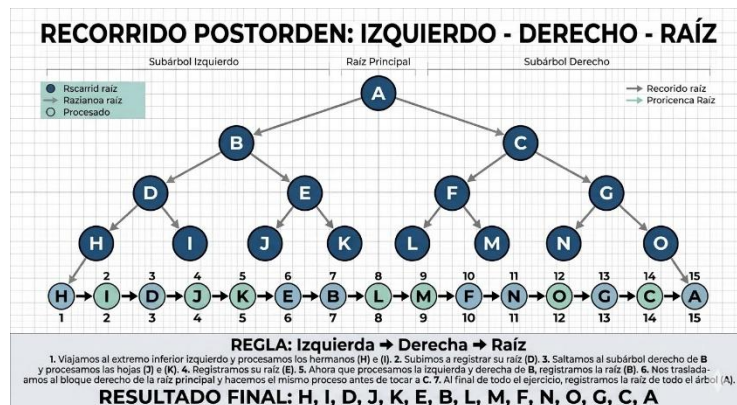
- **Regla:** Izquierda → Raíz → Derecha
- **Explicación del desarrollo:**
 1. Bajamos hasta el fondo izquierdo sin registrar nada hasta tocar el nodo final (**H**).
 2. Subimos a su raíz inmediata (**D**).
 3. Bajamos a su hijo derecho (**I**).
 4. Subimos a la raíz de este bloque completo (**B**).
 5. Pasamos al subárbol derecho de B, buscando su nodo más izquierdo (**J**), luego su raíz (**E**) y su derecho (**K**).
 6. Al terminar el lado izquierdo, procesamos la raíz principal (**A**).
 7. Aplicamos la misma lógica para el subárbol derecho (**C**).
- **Resultado final:** H, D, I, B, J, E, K, A, L, F, M, C, N, G, O



3. Recorrido Postorden (Izquierdo - Derecho - Raíz)

En este recorrido se visita primero el subárbol izquierdo, luego el subárbol derecho y se procesa la raíz al final de cada subárbol. La raíz principal siempre será el último elemento visitado.

- **Regla:** Izquierda → Derecha → Raíz
- **Explicación del desarrollo:**
 1. Viajamos al extremo inferior izquierdo y procesamos los hermanos (**H**) e (**I**).
 2. Subimos a registrar su raíz (**D**).
 3. Saltamos al subárbol derecho de B y procesamos las hojas (**J**) e (**K**).
 4. Registramos su raíz (**E**).
 5. Ahora que procesamos la izquierda y derecha de B, registramos la raíz (**B**).
 6. Nos trasladamos al bloque derecho de la raíz principal y hacemos el mismo proceso antes de tocar a C.
 7. Al final de todo el ejercicio, registramos la raíz de todo el árbol (**A**).
- **Resultado final:** H, I, D, J, K, E, B, L, M, F, N, O, G, C, A



Solución Resumida

A continuación, se presentan los tres recorridos listos para revisión rápida basados en el árbol de 4 niveles:

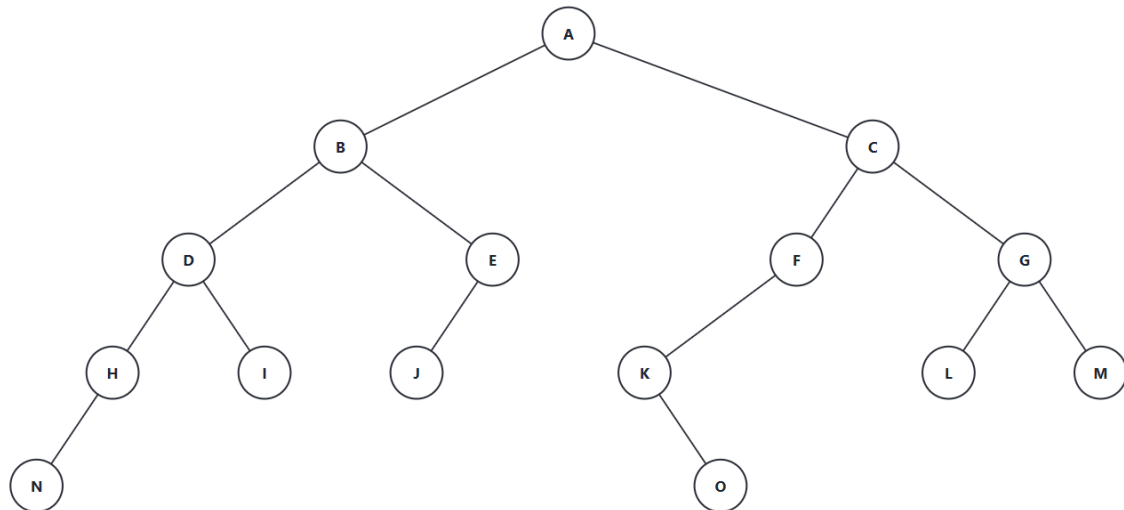
Preorden: A - B - D - H - I - E - J - K - C - F - L - M - G - N - O

Inorden: H - D - I - B - J - E - K - A - L - F - M - C - N - G - O

Postorden: H - I - D - J - K - E - B - L - M - F - N - O - G - C - A

EJERCICIO 4

El Árbol Alfabético Expandido (15 Nodos)



1. Recorrido Preorden (R - I - D)

1. **Raíz principal:** A

2. **Lado Izquierdo (B):**

- Raíz B → Raíz D → Raíz H → Hoja N.
- Regresa a D, ve a la derecha: I.
- Regresa a B, ve a la derecha: Raíz E → Hoja J.

3. **Lado Derecho (C):**

- Raíz C → Raíz F → Raíz K → Hoja O.
- Regresa a C, ve a la derecha: Raíz G → Hoja L, Hoja M.

Resultado Preorden: A, B, D, H, N, I, E, J, C, F, K, O, G, L, M

2. Recorrido Inorden (I - R - D)

1. Extremo izquierdo inferior: N → Raíz H. (H no tiene derecho).
2. Subimos a la raíz del subárbol: D → Derecha de D: I.
3. Subimos a la raíz de todo este bloque: B.
4. Lado derecho de B: Izquierda J → Raíz E.
5. Subimos a la raíz del universo: A.
6. Vamos al bloque derecho (C):
 - En el lado de F: Izquierda K, su derecho O. Raíz F.

- Subimos a la raíz: C.
- Lado derecho de C (G): Izquierda L → Raíz G → Derecha M.

Resultado Inorden: N, H, D, I, B, J, E, A, K, O, F, C, L, G, M

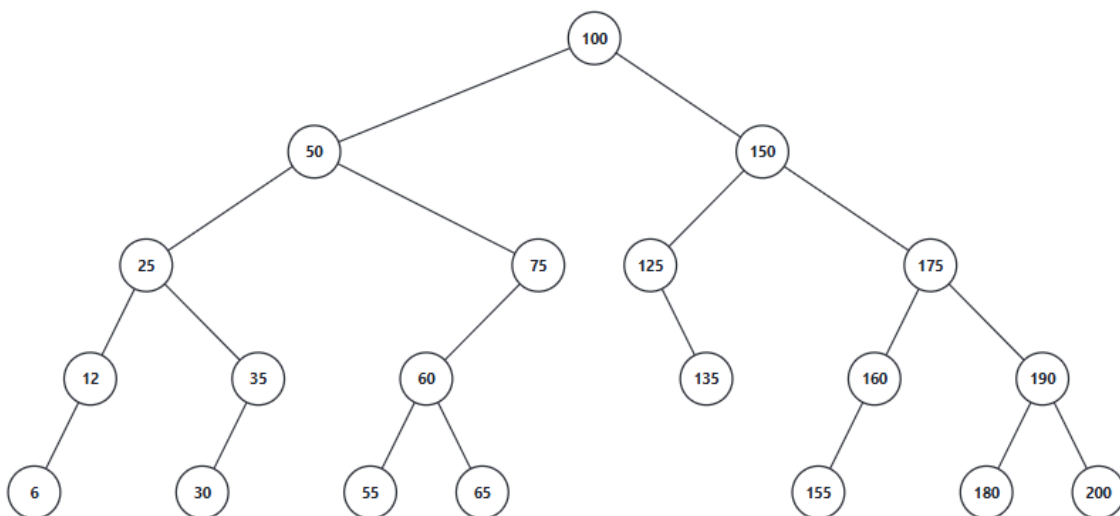
3. Recorrido Postorden (I - D - R)

1. Empezamos desde abajo a la izquierda: Hoja N → Raíz H.
2. Antes de subir a D, procesamos su derecho: I → Raíz D.
3. Antes de subir a B, procesamos su lado derecho (E): Hoja J → Raíz E.
4. Ahora que B tiene sus dos lados listos: Raíz B.
5. Pasamos al lado derecho de la raíz principal (A):
 - En el subárbol de F: El nodo más profundo es derecho de K: O → Raíz K → Raíz F.
 - En el subárbol de G: Hojas L e M → Raíz G.
 - Ya listos los hijos de C, imprimimos la raíz: C.
6. Por último, la corona del árbol: A.

Resultado Postorden: N, H, I, D, J, E, B, O, K, F, L, M, G, C, A

EJERCICIO 5

El Coloso Numérico (20 Nodos - 5 Niveles)



1. Recorrido Preorden (Raíz - Izquierda - Derecha)

Resultado: 100, 50, 25, 12, 6, 35, 30, 75, 60, 55, 65, 150, 125, 135, 175, 160, 155, 190, 180, 200

2. Recorrido Inorden (Izquierda - Raíz - Derecha)

Como es un Árbol Binario de Búsqueda, el inorden **obligatoriamente** se debe listar los números de menor a mayor de forma perfecta.

Resultado: 6, 12, 25, 30, 35, 50, 55, 60, 65, 75, 100, 125, 135, 150, 155, 160, 175, 180, 190, 200

3. Recorrido Postorden (Izquierda - Derecha - Raíz)

No imprimir una raíz hasta haber "vaciado" por completo sus dos ramificaciones inferiores.

Resultado: 6, 12, 30, 35, 25, 55, 65, 60, 75, 50, 135, 125, 155, 160, 180, 200, 190, 175, 150, 100

(APE) – FASE 3

Con el objetivo de mantener la rigurosidad académica, evitar la redundancia documental y optimizar la extensión del presente informe, el equipo de trabajo determinó que el desarrollo detallado de la Fase 3 de la APE se encuentra consolidado mediante un repositorio digital externo.

Dado que los ejercicios prácticos y los diagramas estructurales de árboles exigidos para esta fase coinciden en su totalidad con el diseño metodológico desarrollado en las secciones previas de este portafolio, se ha evitado la duplicación inútil de datos para garantizar un documento limpio, fluido y de alta calidad técnica.

<https://drive.google.com/file/d/1uAndiCsc4L4Ch8mq1kvucyb2KKZpH8g/view?usp=sharing>

PORTAFOLIOS

Jaramillo, Y. (2026). Portafolio de Matemáticas Discretas [Repositorio de GitHub].

GitHub. <https://github.com/yuleisyjaramillo2008/Matem-ticas-Discretas>

Nero, J. (2026). Portafolio de Matemáticas Discretas [Repositorio de GitHub].

GitHub. <https://github.com/josselinnero-svg/Portafolio-Matem-ticas-Discretas/blob/main/index.md>

Guevara, W. (2026). Matemáticas Discretas [Repositorio de GitHub].

GitHub. <https://github.com/moncadajoelwg-del/matematicasDiscretas/blob/main/MATE-DISCRETAS.md>

Roblez, S. (2026). Discrete math portfolio [Repositorio de GitHub].

GitHub. <https://github.com/sdierrobles-lgtm/discrete-math-portfolio>

CONCLUSION

La teoría de árboles y sus métodos de recorrido demuestran que la optimización y la velocidad de los sistemas informáticos actuales dependen directamente de una organización matemática estructurada. Al definir los árboles como grafos conexos sin ciclos, se garantiza la existencia de un único camino entre cualquier par de nodos, transformando la gestión masiva de datos en un proceso predecible y eficiente.

A través de estructuras específicas, como los Árboles de Búsqueda Binaria (ABB) y los árboles balanceados AVL, la informática logra reducir búsquedas complejas a tiempos logarítmicos $O(\log n)$ mediante decisiones binarias consecutivas. Asimismo, la implementación de los recorridos en profundidad preorden, inorden y postorden junto con técnicas de optimización como el Código

Huffman, prueban que el verdadero valor de estas arquitecturas jerárquicas no radica únicamente en almacenar información. Radica en la versatilidad operativa que ofrecen para resolver problemas del mundo real, permitiendo a los desarrolladores duplicar estructuras, extraer reportes ordenados de menor a mayor, comprimir archivos o liberar memoria física de forma segura de abajo hacia arriba.

REFLEXION PERSONAL

Yuleisy Cecibel Jaramillo:

En el área de la computación, estas estructuras dejan de ser simples diagramas teóricos y se convierten en herramientas clave para resolver problemas del mundo real. Al inicio, conceptos como "árbol enraizado" o "longitud de paseo" parecen solo definiciones abstractas de un libro, pero conforme se avanza en su aplicación práctica, uno entiende que son la base invisible detrás de cosas tan cotidianas como buscar un archivo o cargar una página web en segundos. Trabajar con los recorridos preorden, inorden y postorden me hizo ver que no basta con organizar la información correctamente; también es necesario saber *cómo* recorrerla, porque cada método responde a una necesidad distinta: ordenar, clonar o eliminar una estructura de datos. Finalmente, comprender la lógica de los recorridos de árboles entrena la mente para la resolución de problemas complejos y el diseño de algoritmos, y me deja claro que estas herramientas matemáticas serán una parte constante de mi formación como futura ingeniera en computación.

Sdier Emanuel Roblez.

El famoso matemático John von Neumann decía una frase muy real: *"En matemáticas uno no entiende las cosas, se acostumbra a ellas"*. Al principio, cuando escuchamos nombres como "Árboles AVL" o "conjuntos de corte", nos da dolor de cabeza. Pero cuando entiendes que solo sirven para conectar cosas y ahorrar tiempo, dejas de verlos como fórmulas difíciles. Te acostumbras a verlos como lo que son: una forma muy inteligente y ordenada que tiene nuestra mente para resolver problemas complicados de la manera más sencilla posible.

Wilson Joel Guevara Moncada

El análisis procedimental de los recorridos en profundidad me ha permitido ver el software desde una perspectiva interna y operativa. Aprender que un recorrido en inorden sirve para generar reportes limpios, o que el postorden es el estándar lógico para que los sistemas operativos liberen memoria física de forma segura, conecta directamente la teoría matemática con el mantenimiento de sistemas en el mundo real. Este conocimiento me da herramientas críticas para optimizar el rendimiento de los procesadores y gestionar el almacenamiento de manera estratégica

Josselin Estefania Nero Fernandez

Las estructuras de árbol son fundamentales en la informática porque permiten organizar y

procesar datos jerárquicos de manera eficiente, lo cual resulta esencial para la búsqueda rápida de información y la representación de relaciones lógicas. En nuestra vida cotidiana, las vemos en acción en los sistemas de directorios de nuestras computadoras, en la organización de los menús en las aplicaciones y sitios web, y en la estructura de las bases de datos que soportan los servicios que usamos a diario.

REFERENCIAS

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.^a ed.). MIT Press.
- Joyanes Aguilar, L. (2020). *Fundamentos de programación: Algoritmos, estructura de datos y objetos* (5.^a ed.). McGraw-Hill.
- Rosen, K. H. (2019). *Discrete mathematics and its applications* (8.^a ed.). McGraw-Hill.
- Villalpando Becerra, J. F., & García Sandoval, A. (2014).** [*Matemáticas discretas: Aplicaciones y ejercicios*](#). Grupo Editorial Patria.
- Weiss, M. A. (2014). *Data structures and algorithm analysis in C++* (4.^a ed.). Pearson.